BUỔI 5

# Dữ liệu chuỗi, embedding và RNN/LSTM

- Hiểu đặc thù dữ liệu chuỗi và văn bản.
- Nắm embedding, RNN, LSTM và vấn đề gradient theo thời gian.
- Thực hành phân loại văn bản với LSTM.
- Biết xử lý padding đúng cách.

# MINH HỌA • Dữ liệu chuỗi, embedding và RNN/LSTM

SƠ ĐỒ / HÌNH MINH HỌA




Thứ tự và ngữ cảnh của chuỗi ảnh hưởng đến kết quả

## VÍ DỤ THỰC TẾ

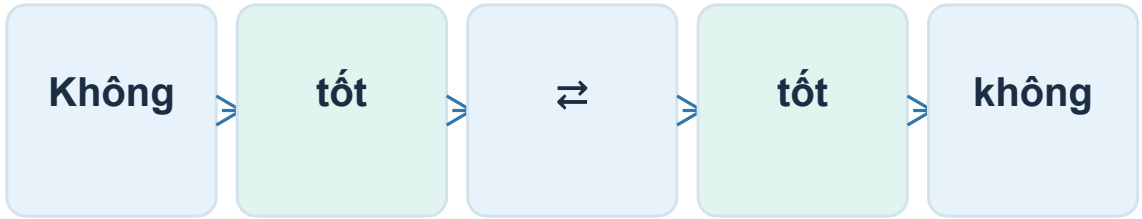
Phân loại tin tức AG News cần xem cả chuỗi từ, không chỉ từng từ độc lập.

# Dữ liệu chuỗi khác ảnh như thế nào?

- Ảnh có cấu trúc không gian; chuỗi có thứ tự thời gian hoặc thứ tự token.
- Ý nghĩa của một phần tử phụ thuộc vào phần tử trước/sau nó.
- Độ dài chuỗi thường không cố định, cần padding hoặc batching đặc biệt.
- Ví dụ: câu văn, chuỗi giá cổ phiếu, log hệ thống, tín hiệu cảm biến.
- Mô hình cần học cả nội dung từng token và quan hệ theo thứ tự.

# MINH HỌA • Dữ liệu chuỗi khác ảnh như thế nào?

SƠ ĐỒ / HÌNH MINH HỌA




Thứ tự và ngữ cảnh của chuỗi ảnh hưởng đến kết quả

## VÍ DỤ THỰC TẾ

“Không tốt” khác “tốt”: đổi thứ tự và ngữ cảnh có thể làm đảo ngược sắc thái.

# Tokenization và vocabulary

| Khái niệm | Nội dung |
|---|---|
| Tokenization | Tách văn bản thành các đơn vị nhỏ (token), có thể là từ, từ con hoặc ký tự. |
| Vocabulary | Bộ từ vựng ánh xạ mỗi token thành một số nguyên (ID) để mô hình xử lý. |
| <unk> | Ký hiệu thay thế token không có trong vocabulary, thường dùng trong các mô hình có từ vựng cố định. |
| Padding | Thêm token đặc biệt, thường là <pad>, để các chuỗi trong batch có cùng độ dài. |
| Truncation | Cắt bớt token khi chuỗi dài hơn giới hạn cho phép. |




Tokenization chia văn bản → Vocabulary đổi thành số → Padding/Truncation chuẩn hóa độ dài → Mô hình học sâu xử lý.

# Embedding

- Embedding biến token id thành vector liên tục có thể học được.
- Các token có ngữ cảnh/ý nghĩa gần nhau có thể có vector gần nhau trong không gian embedding.
- Embedding giúp mô hình học biểu diễn thay vì xử lý id rời rạc.
- Kích thước embedding là hyperparameter quan trọng.
- Embedding quá nhỏ thiếu năng lực biểu diễn, quá lớn dễ tăng chi phí và overfit.




Các từ có thể có vị trí gần/xa trong không gian vector

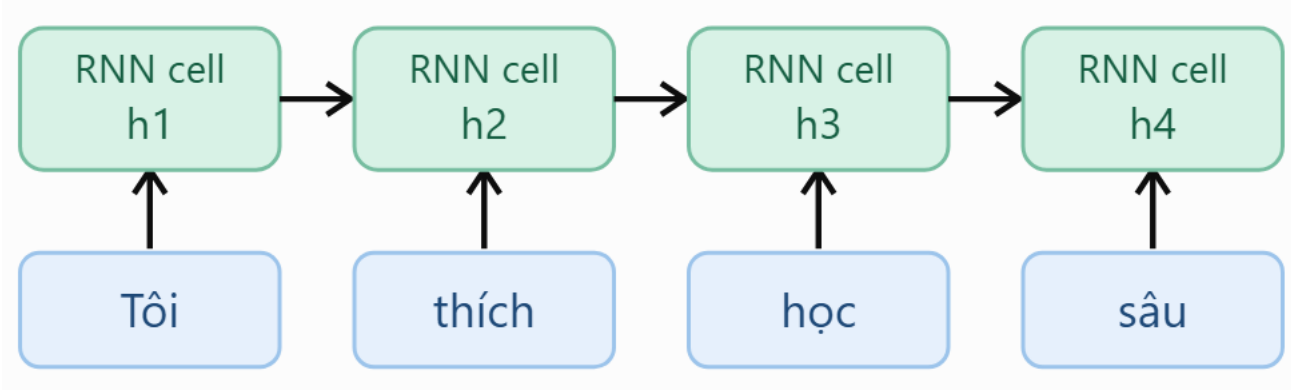
## VÍ DỤ

"bóng đá" và "cầu thủ" có thể được học thành các vector gần nhau từ ngữ cảnh tin thể thao.

# RNN: mô hình có bộ nhớ theo thời gian

• RNN (Recurrent Neural Network) là mạng nơ-ron hồi quy, được thiết kế để xử lý dữ liệu dạng chuỗi như văn bản hoặc âm thanh.

| Khái niệm | Nghĩa là |
|---|---|
| Xử lý từng bước | RNN đọc lần lượt từng token trong câu, theo thứ tự. |
| Hidden state $h_t$ | Bộ nhớ tạm, lưu thông tin tổng hợp từ các token đã đọc đến thời điểm t |
| Chia sẻ tham số | Cùng một RNN cell và bộ trọng số được sử dụng cho tất cả các bước thời gian. |
| Học thông tin chuỗi | Kết hợp token hiện tại với thông tin từ các token trước để hiểu ngữ cảnh. |
| Hạn chế | Khó ghi nhớ thông tin cách xa nhiều bước vì gradient có thể quá nhỏ hoặc quá lớn khi huấn luyện. |




Mỗi cell nhận token hiện tại và hidden state từ bước trước, rồi tạo hidden state mới.

- Đọc "Tôi" → ghi nhớ thông tin ban đầu.
- Đọc "thích" → cập nhật thành ngữ cảnh "Tôi thích".
- Đọc "học sâu" → cập nhật bộ nhớ để biểu diễn thông tin của cả câu.

RNN giống một người đọc từng từ và liên tục cập nhật trí nhớ. Tuy nhiên, **câu càng dài thì thông tin từ đầu câu càng dễ bị quên.**

# LSTM: kiểm soát dòng thông tin

- LSTM (Long Short-Term Memory) thêm cell state và các cổng để quyết định giữ, quên hoặc ghi thông tin.
- Forget gate quyết định thông tin cũ nào được giữ lại.
- Input gate quyết định thông tin mới nào được ghi vào cell state.
- Output gate quyết định phần nào của trạng thái được đưa ra hidden state.
- LSTM cải thiện khả năng học phụ thuộc dài so với RNN cơ bản.

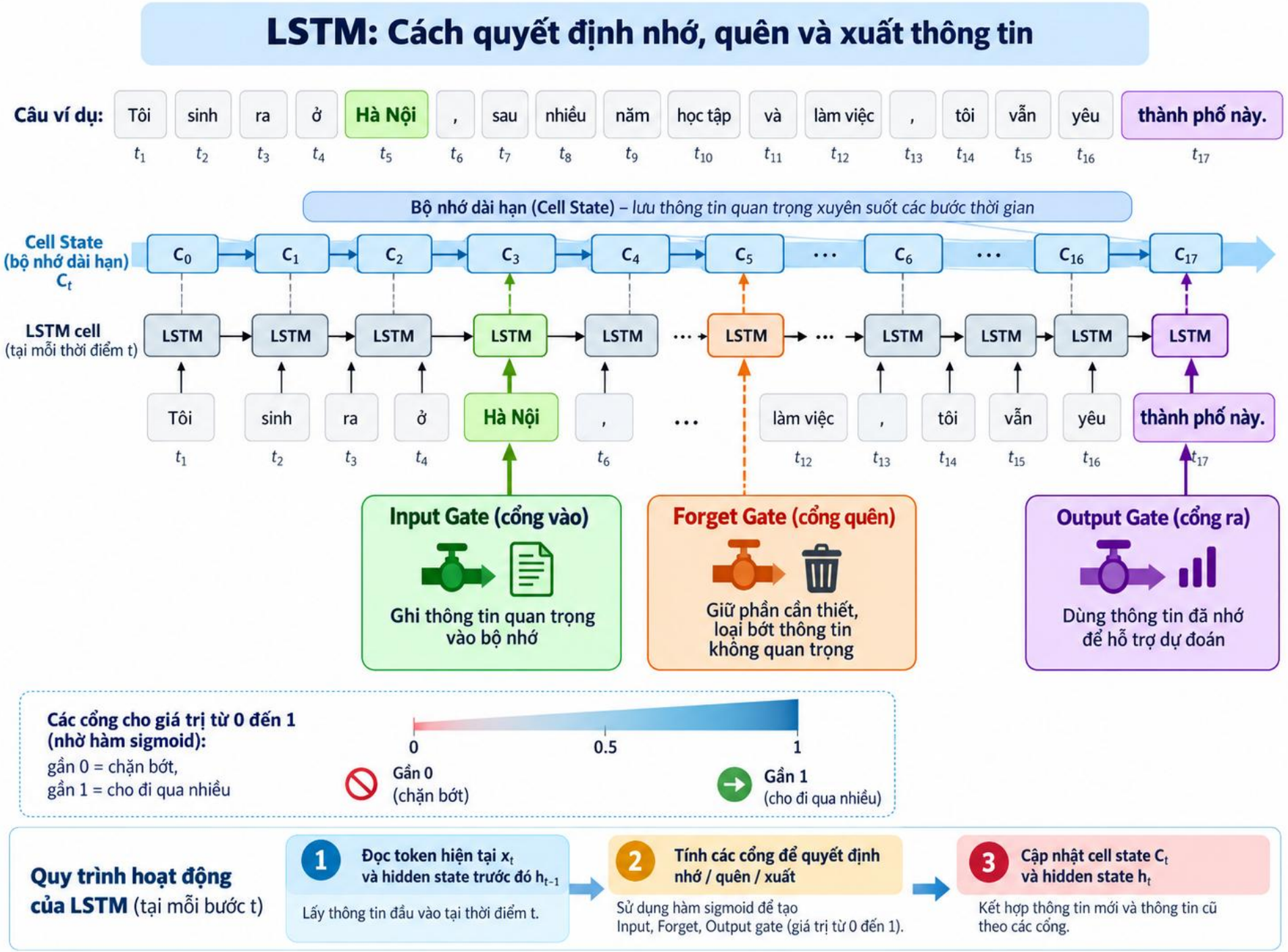
*Tôi sinh ra ở Hà Nội, sau nhiều năm học tập và làm việc, tôi vẫn yêu thành phố này*

- Forget Gate: Loại bỏ bớt những thông tin không cần thiết.
- Input Gate: Ghi nhớ thông tin quan trọng như địa danh Hà Nội.
- Cell State: Duy trì thông tin Hà Nội qua nhiều từ.
- Output Gate: Sử dụng thông tin đã ghi nhớ để hỗ trợ hiểu rằng "thành phố này" nhắc đến Hà Nội.

LSTM tự học cách quyết định nhớ, quên hay duy trì thông tin thông qua các trọng số được tối ưu trong quá trình huấn luyện

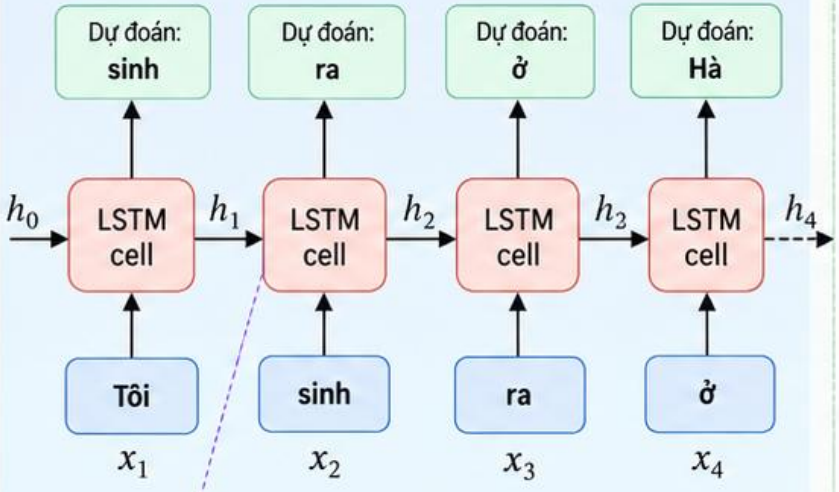
Trong quá trình huấn luyện:

1. LSTM dự đoán kết quả từ dữ liệu.
2. Tính Loss để đo mức sai lệch.
3. Dùng Backpropagation Through Time (BPTT) để tính gradient.
4. Optimizer cập nhật trọng số của các cổng nhằm giảm Loss.

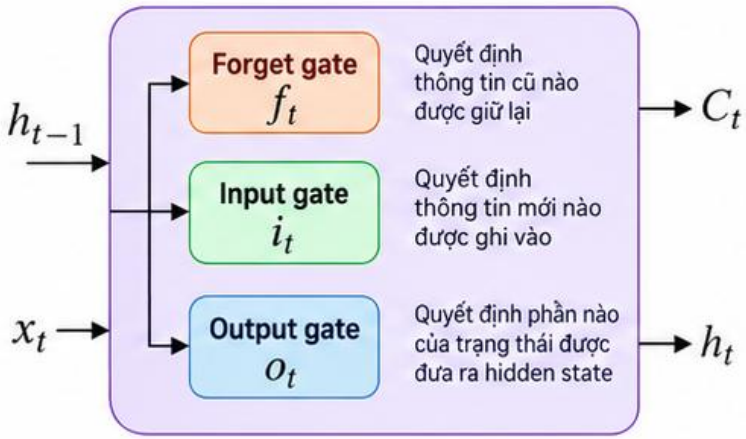


## 1 Dự đoán kết quả từ dữ liệu (Forward pass)

LSTM đọc chuỗi từng token và dự đoán token tiếp theo tại mỗi bước.




### Bên trong một LSTM cell (tại bước t)



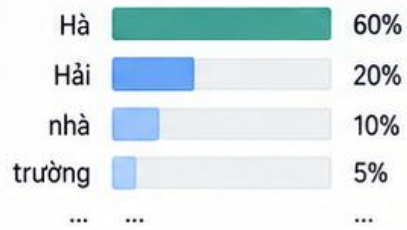

## 2 Tính Loss (đo sai lệch)

So sánh xác suất dự đoán với token đúng tại mỗi bước.

**Ví dụ tại bước t = 4**

| Token | Xác suất (dự đoán) | Token đúng |
|---|---|---|
| Hà | 0,60 | 1 |
| Hải | 0,20 | 0 |
| nhà | 0,10 | 0 |
| trường | 0,05 | 0 |
| ... | ... | ... |

**Phân phối xác suất (Softmax)**




**Loss tại bước t**

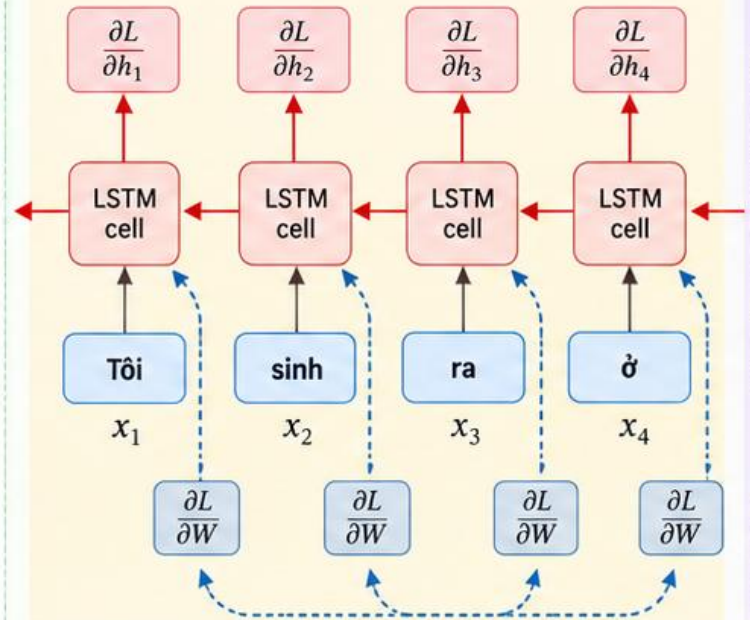
$$L_t = -\log P(\text{“Hà”}) = -\log(0{,}60) \approx 0{,}51$$

**Tổng Loss cho cả chuỗi**

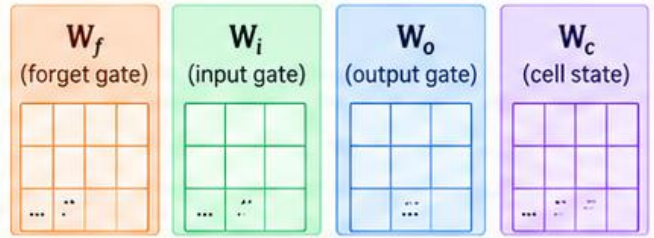
$$L = \frac{1}{T}\sum_{t=1}^{T} L_t$$

## 3 Backpropagation Through Time (tính gradient)

Truyền gradient ngược qua tất cả các bước thời gian.




**Gradient được tính cho các tham số của LSTM**




Mỗi tham số nhận gradient từ tất cả các bước thời gian vì cùng một cell được lập lại (chia sẻ tham số).

## 4 Cập nhật trọng số (giảm Loss)

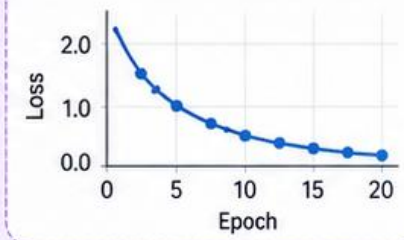
Optimizer (ví dụ: Adam, SGD) cập nhật các tham số bằng gradient để giảm Loss.

**Cập nhật tham số (ví dụ Adam)**

$$W \leftarrow W - \eta \cdot \text{gradient}$$

- $\eta$: learning rate
- gradient: được tính bằng BPTT
- Cập nhật cho tất cả các tham số: $W_f, W_i, W_o, W_c, \ldots$

**Loss giảm dần qua các epoch**




**Mô hình học tốt hơn**

- Dự đoán chính xác hơn
- Các cổng học cách giữ, quên, ghi thông tin phù hợp
- Hiểu ngữ cảnh dài tốt hơn

# Padding không phải token thật

- Batch văn bản cần padding để các câu có cùng độ dài.
- Nếu lấy hidden state cuối sau padding, mô hình có thể học từ token giả.
- Cần dùng length thật hoặc pack_padded_sequence để bỏ qua padding.
- Metric cũng cần đảm bảo không tính nhầm phần padding trong bài toán token-level.

# MINH HỌA • Padding không phải token thật

SƠ ĐỒ / HÌNH MINH HỌA



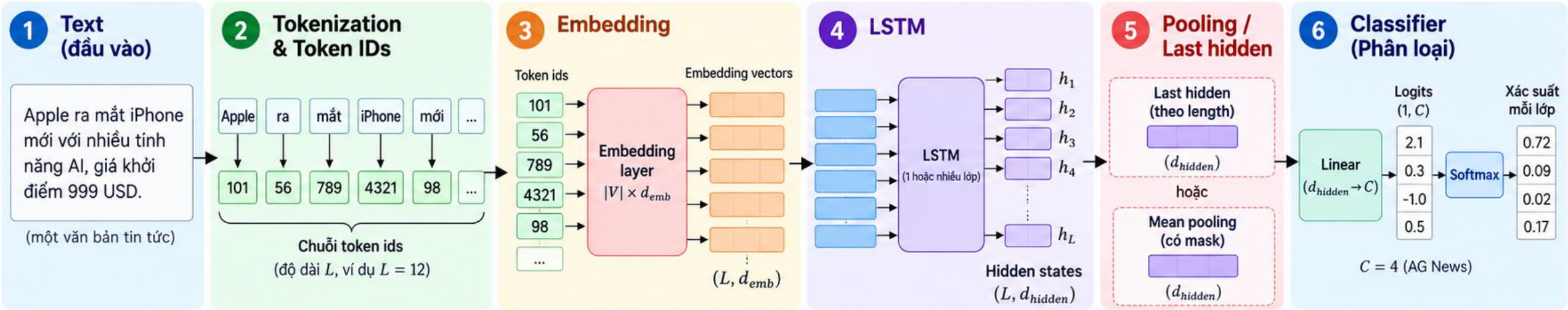

Mô hình phải bỏ qua vị trí <pad> khi xử lý chuỗi

## VÍ DỤ THỰC TẾ

Câu 5 từ được đệm thành 12 token; phần đệm không được tính như 7 từ thật.

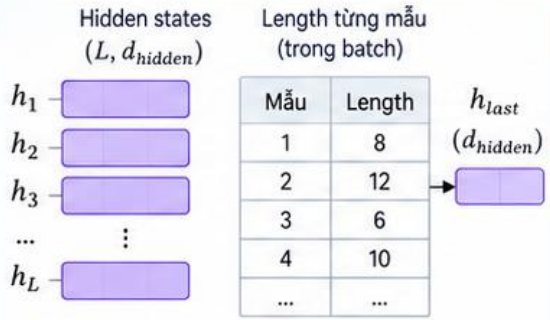
# Phân loại văn bản bằng LSTM

- Pipeline: text → token ids → embedding → LSTM → pooling/last hidden → classifier.
- Bài toán phân loại dùng logits cho từng lớp văn bản.
- Có thể dùng hidden state cuối đúng theo length hoặc mean pooling có mask.
- Dataset AG News là ví dụ phù hợp: phân loại tin tức thành 4 chủ đề.
- Cần kiểm tra vocabulary size, sequence length và batch shape.



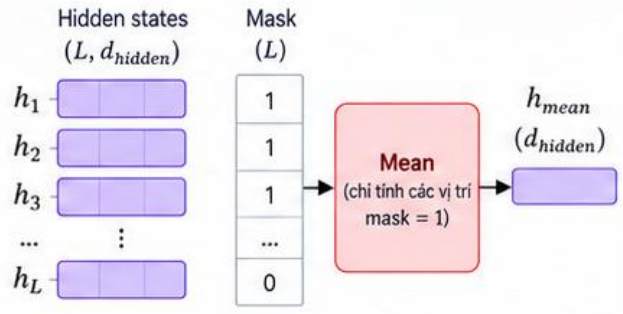

## A. Lấy biểu diễn từ LSTM

### A1. Last hidden (theo độ dài thực)



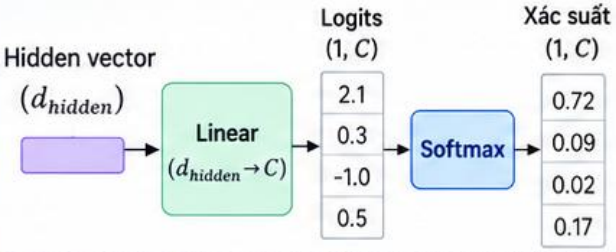

Lấy hidden state tại vị trí cuối cùng theo length của từng mẫu (bỏ qua padding).

### A2. Mean pooling (có mask)




Lấy trung bình hidden states tại các vị trí không phải padding (dựa trên *mask*).

## B. Phân loại với logits




**Các lớp của AG News ($C = 4$)**

- 0 **World** (Thế giới)
- 1 **Sports** (Thể thao)
- 2 **Business** (Kinh doanh)
- 3 **Sci/Tech** (Khoa học - Công nghệ)

## C. Ví dụ về AG News

Mỗi văn bản là một tin tức, phân loại thành 4 chủ đề.

| Văn bản (rút gọn) | Nhãn |
|---|---|
| World leaders meet at climate summit to discuss global warming... | 0 World |
| The team won the championship after a thrilling final match... | 1 Sports |
| Apple reports strong quarterly earnings as iPhone sales grow... | 2 Business |
| New AI model shows impressive performance on benchmark tests... | 3 Sci/Tech |

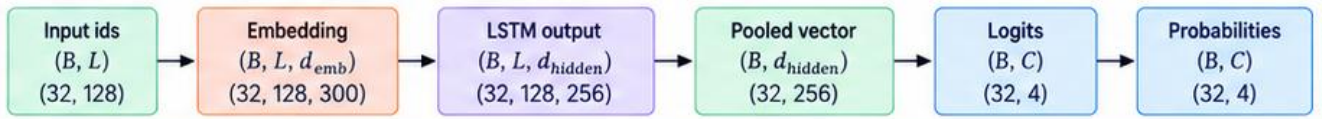
## D. Các tham số cần kiểm tra và shapes trong batch

**Vocabulary size |V|**
- Số token trong từ vựng
- Ví dụ: $|V| = 30{,}000$

**Sequence length L**
- Độ dài tối đa của chuỗi (sau padding/cắt)
- Ví dụ: $L = 128$

**Batch size B**
- Số mẫu trong một batch
- Ví dụ: $B = 32$

**Các shape** (với $B$=32, $L$=128, $d_{emb}$=300, $d_{hidden}$=256, $C$=4)



# Ưu điểm và hạn chế của RNN/LSTM

| Nội dung | Nghĩa là |
|---|---|
| Xử lý chuỗi tự nhiên | RNN/LSTM đọc từng token theo thứ tự, kết hợp thông tin mới với thông tin đã ghi nhớ. |
| Số tham số không tăng theo độ dài chuỗi | Cùng một bộ trọng số được sử dụng lại ở mọi bước thời gian. Câu có 10 hay 100 token vẫn dùng cùng bộ tham số. |
| Dễ hình dung | Giống người đọc một câu từ trái sang phải, vừa đọc vừa cập nhật trí nhớ. |
| Khó song song hóa | Muốn xử lý token thứ 5, RNN/LSTM phải có hidden state từ token thứ 4, nên không thể tính đồng thời tất cả bước thời gian như Transformer thông thường. |
| Khó học phụ thuộc xa | Với câu rất dài, thông tin ở đầu câu có thể bị suy giảm hoặc không được duy trì đầy đủ đến cuối câu. |

Câu: “Tôi sinh ra ở Hà Nội, sau nhiều năm học tập và làm việc ở nhiều nơi, tôi vẫn yêu thành phố này.”

LSTM: Đọc lần lượt từng token và cố duy trì thông tin “Hà Nội” trong bộ nhớ để hiểu “thành phố này”.

Transformer: Sử dụng cơ chế Self-Attention, cho phép biểu diễn của “thành phố này” kết hợp trực tiếp thông tin từ “Hà Nội”, không cần truyền tuần tự qua tất cả token trung gian

**BUỔI 5 – THỰC HÀNH LSTM PHÂN LOẠI VĂN BẢN**

**1. Bài thực hành trên lớp: Phân loại tin tức bằng LSTM**

**Mục tiêu:** Xây dựng và huấn luyện LSTM phân loại văn bản AG News thành 4 chủ đề: World, Sports, Business, Sci/Tech.

**Yêu cầu:**

- Sử dụng file bai05_class_lstm_text.py và tập con AG News.
- Thực hiện tokenization, xây dựng vocabulary, chuyển token thành ID và padding chuỗi.
- Kiểm tra vocabulary size, sequence length, embedding shape và batch shape.
- Xây dựng mô hình **Embedding → LSTM → Linear**, sử dụng pack_padded_sequence để xử lý padding đúng.
- Huấn luyện và báo cáo train loss, validation accuracy theo từng epoch; phân tích ít nhất **3 câu dự đoán sai**.

**Sản phẩm nộp:** Mã nguồn chạy được, bảng kết quả, learning curve và nhận xét ngắn.

**2. Bài tập về nhà: So sánh các biến thể LSTM**

**Mục tiêu:** Đánh giá ảnh hưởng của cấu hình LSTM đến chất lượng phân loại văn bản AG News.

**Yêu cầu theo mức độ:**

- **Cơ bản:** Huấn luyện LSTM baseline; báo cáo validation accuracy, Macro-F1 và loss.
- **Khá:** Thay đổi embedding_dim hoặc hidden_dim; so sánh với baseline trong cùng điều kiện huấn luyện.
- **Nâng cao:** Thử **Bidirectional LSTM (BiLSTM)** hoặc **Dropout**; so sánh độ chính xác, số tham số và thời gian chạy.
- **Đánh giá:** Vẽ learning curve, lập confusion matrix và phân tích ít nhất **5 câu dự đoán sai**, giải thích nguyên nhân.

**Sản phẩm nộp:** Mã nguồn, bảng so sánh kết quả, biểu đồ, ma trận nhầm lẫn và phân tích lỗi.

# Tóm tắt buổi 5

- Dữ liệu chuỗi cần mô hình hiểu thứ tự và ngữ cảnh.
- Embedding chuyển token rời rạc thành vector liên tục có thể học.
- RNN/LSTM dùng hidden state để truyền thông tin theo thời gian.
- Padding cần được xử lý đúng bằng length/mask.
- Buổi 6 học attention và Transformer để giải quyết ngữ cảnh hiệu quả hơn.